

P2P Car Rental System

Final Report

Course	MSE800 Professional Software Engineering
Institution	Yoobee College of Creative Innovation
Assignment	Assignment 1 - Object-Oriented Programming (30%)
Document	Final Report
Version	1.0.0
Date	May 2026
Author	[Your Name]

A car rental system built in Python. The system uses object-oriented programming, design patterns, and a layered architecture.

Abstract

The P2P Car Rental System is a command-line application written in Python. The system connects car owners with renters. Three types of users can use the system: Owner, Renter, and Admin. Each user has a different menu. The platform takes a 15% commission on each booking.

The system uses the four main ideas of object-oriented programming: encapsulation, inheritance, abstraction, and polymorphism. It also uses three design patterns: Singleton, Factory, and Strategy. The data is stored in an SQLite database. The project has 8 UML diagrams and a separate maintenance plan.

Keywords:

Object-Oriented Programming, Design Patterns, Python, SQLite, Peer-to-Peer, UML, Singleton, Factory, Strategy, CLI.

Table of Contents

#	Section	Page
1	Introduction	4
2	System Requirements	5
3	Installation	6
4	Configuration	7
5	User Guide	8
6	Features	10
7	Project Structure	11
8	System Background (Screenshots)	12
9	Design Patterns	15
10	UML Diagrams	17
11	Testing	25
12	Maintenance Summary	27
13	Conclusion	28
	References	29
	Appendix: Default Credentials	30

1. Introduction

1.1 Project Background

The car rental market has changed in recent years. New companies like Turo use a peer-to-peer (P2P) model. In this model, normal people rent out their own cars to other people. The platform does not own the cars. It only connects the owner and the renter. This is similar to how Airbnb works for houses.

The P2P model has some benefits. Owners can earn money from a car that is not used every day. Renters can choose from many different cars in their city. Prices are often lower than traditional rental companies.

1.2 Problem Statement

A P2P platform needs to solve some problems:

- Trust: how do we know users are real people, not scammers?
- Quality: how do we keep low-quality or fake cars off the platform?
- Booking: how do we handle dates, prices, and approvals?
- Reviews: how do we build trust between users after a rental?
- Damage: how do we handle insurance claims when a car is damaged?

1.3 Project Scope

This project is a working Python CLI application. It covers these features: user registration, KYC verification, car listing, search and filter, booking with insurance and discount, owner approval, reviews, and insurance claims. The system is for educational use. It does not connect to real payment systems or send real emails. But the design is ready for these features in the future.

1.4 Objectives

Objective	How we achieve it
Show OOP principles	Abstract User class with Owner, Renter, and Admin subclasses. Private attributes and a polymorphic dashboard method.
Use design patterns	Singleton for the database, Factory for users, Strategy for insurance.
Use a layered architecture	Four layers: Presentation, Service, Model, and Data Access.
Handle errors well	Input validators, try/except for database calls, and parameterized SQL.
Support different environments	config.py reads the APP_ENV variable to switch between dev, test, and prod.
Document with UML	8 UML diagrams: ERD, Use Case, Class, Sequence, and 4 Activity diagrams.

2. System Requirements

2.1 Hardware Requirements

Component	Minimum	Recommended
CPU	1 GHz, any modern CPU	2 GHz or faster, dual-core
RAM	512 MB	2 GB
Disk	50 MB free	500 MB free
Display	80-column terminal	Modern terminal emulator

2.2 Software Requirements

Software	Version	Purpose
Python	3.8 or higher	Programming language.
pip	Built-in	Package installer.
SQLite	Built into Python	Database (no separate install).
tabulate	0.9.0	Print pretty tables in the CLI.
colorama	0.4.6	Add colors to the CLI output.
Operating System	Windows 10+, macOS 10.14+, or Linux	Cross-platform support.

Note: You do not need a separate database server. SQLite saves all data in one file called `database/car_rental.db`. The file is created automatically on the first run.

3. Installation

Installation takes about two minutes. Follow these three steps.

Step 1: Extract the zip file

Extract **p2p_car_rental.zip** to any folder. The extracted folder is the project root.

Step 2: Create a virtual environment

```
# Create the virtual environment
python -m venv venv

# Activate it (Windows)
venv\Scripts\activate

# Activate it (macOS or Linux)
source venv/bin/activate
```

Step 3: Install dependencies

```
# Run this from the project folder, with venv active:
pip install -r requirements.txt
```

This installs tabulate and colorama. Now the system is ready to run.

3.1 Running the Application

```
python main.py
```

On the first run, the system does these things automatically:

- Creates the SQLite database file (database/car_rental.db).
- Sets up all tables and constraints.
- Creates the default admin account.
- Shows the main menu (Login, Register, Exit).

4. Configuration

All settings are in the **config.py** file. The active environment is selected by the *APP_ENV* environment variable. If no variable is set, the system uses dev.

4.1 Environments

Environment	Database	Purpose
dev (default)	database/car_rental.db	For development. Data is saved between runs.
test	:memory: (in-RAM SQLite)	For unit tests. Data is deleted on every run.
prod	database/car_rental_prod.db	For production use.

4.2 How to Change Environment

```
# macOS or Linux
export APP_ENV=test
python main.py

# Windows (CMD)
set APP_ENV=test
python main.py

# Or inline (Linux/macOS)
APP_ENV=test python main.py
```

4.3 Configuration Values

Constant	Default	Description
PLATFORM_FEE_PERCENT	15%	Commission per booking.
LONG_TERM_DISCOUNTS	7d=10%, 30d=20%	Discount tiers for long rentals.
DISCOUNT_CODES	SAVE10, WELCOME20, SUMMER15	Demo discount codes.
DEFAULT_ADMIN_EMAIL	admin@p2p.com	Default admin email.
DEFAULT_ADMIN_PASSWORD	admin123	Default admin password.

5. User Guide

The system has three user roles. Each role has a different menu. For first-time users, this is a good order to test all features:

#	Step	Role
1	Login as admin (admin@p2p.com / admin123).	Admin
2	Register a new owner account, then logout.	Anonymous
3	Register a new renter account, then logout.	Anonymous
4	Login as admin and verify both new users.	Admin
5	Login as owner and add a car listing.	Owner
6	Login as admin and approve the car.	Admin
7	Login as renter, search cars, and make a booking.	Renter
8	Login as owner and approve the booking.	Owner
9	Login as renter and submit a review.	Renter

5.1 Owner Features

- **Add car listing:** Enter make, model, year, mileage, location, and daily rate. The listing waits for admin approval.
- **Manage bookings:** See pending requests, then approve or reject each.
- **View earnings:** See total income from completed bookings.

5.2 Renter Features

- **Search cars:** Filter by location and maximum price per day.
- **View details:** See full car information and rental period.
- **Make booking:** Choose dates, select insurance, and add a discount code if you have one.
- **Cancel booking:** You can cancel before the rental starts.
- **Submit review:** Rate the owner from 1 to 5 stars after the rental.

5.3 Admin Features

- **Verify users:** Check that new users are real people (KYC).
- **Approve cars:** Reject low-quality or suspicious listings.
- **View bookings and reports:** See all bookings and platform statistics.
- **Handle claims:** Decide insurance claims (approve, partial, reject).
- **Suspend users:** Remove verification for users who break the rules.

5.4 Discount Codes

Code	Discount
SAVE10	10% off the base price
WELCOME20	20% off the base price
SUMMER15	15% off the base price

6. Features

The system has these main features, grouped by role:

Category	Features
Authentication	User registration, login, password hashing (SHA-256 with salt), session management.
Owner features	Add and remove cars. Set price and availability. Approve or reject bookings. View earnings.
Renter features	Search and filter cars. View car details. Make and cancel bookings. Submit reviews.
Admin features	Verify users. Approve or reject car listings. View all bookings. Handle insurance claims. Suspend users.
Pricing engine	Daily rate times days. Long-term discounts (7+ days = 10%, 30+ days = 20%). Optional discount codes. Optional insurance. 15% platform fee.
Insurance plans	Three options: Basic (\$5/day), Standard (\$10/day), Premium (\$15/day). Uses the Strategy design pattern.
Reviews and ratings	Renters rate owners from 1 to 5 stars after a booking. Average rating is shown on the dashboard.
Reports	Admin can see total users, owners, renters, cars, bookings, and platform revenue.

7. Project Structure

The code follows a 4-layer architecture. Each folder is one layer or one helper module.

```
p2p_car_rental/
|
|-- main.py # Entry point
|-- config.py # Configuration
|-- requirements.txt # Python dependencies
|-- README.md # User documentation
|
|-- database/ # Data Access Layer
|   |-- db_manager.py # Singleton DB manager
|   |-- car_rental.db # SQLite file (auto-created)
|
|-- models/ # Domain (OOP entities)
|   |-- user.py # Abstract User class
|   |-- owner.py # Inherits User
|   |-- renter.py # Inherits User
|   |-- admin.py # Inherits User
|   |-- car.py # Car entity
|   |-- booking.py # Booking entity
|   |-- review.py # Review entity
|
|-- factories/ # Factory pattern
|   |-- user_factory.py
|
|-- patterns/ # Design patterns
|   |-- insurance_strategy.py # Strategy pattern
|
|-- services/ # Business logic layer
|   |-- auth_service.py
|   |-- car_service.py
|   |-- booking_service.py
|   |-- review_service.py
|   |-- admin_service.py
|
|-- ui/ # Presentation (CLI menus)
|   |-- main_menu.py
|   |-- owner_menu.py
|   |-- renter_menu.py
|   |-- admin_menu.py
|
|-- utils/ # Helpers
|   |-- security.py # Password hashing
|   |-- validators.py # Input validators
|   |-- helpers.py # CLI display helpers
|
|-- docs/ # Documentation
|-- UML/ # 8 UML diagrams (PNG)
|-- screenshots/ # CLI screenshots
```

8. System Background (Screenshots)

These screenshots show the system in action. They show the actual CLI for each role.

8.1 Main Menu

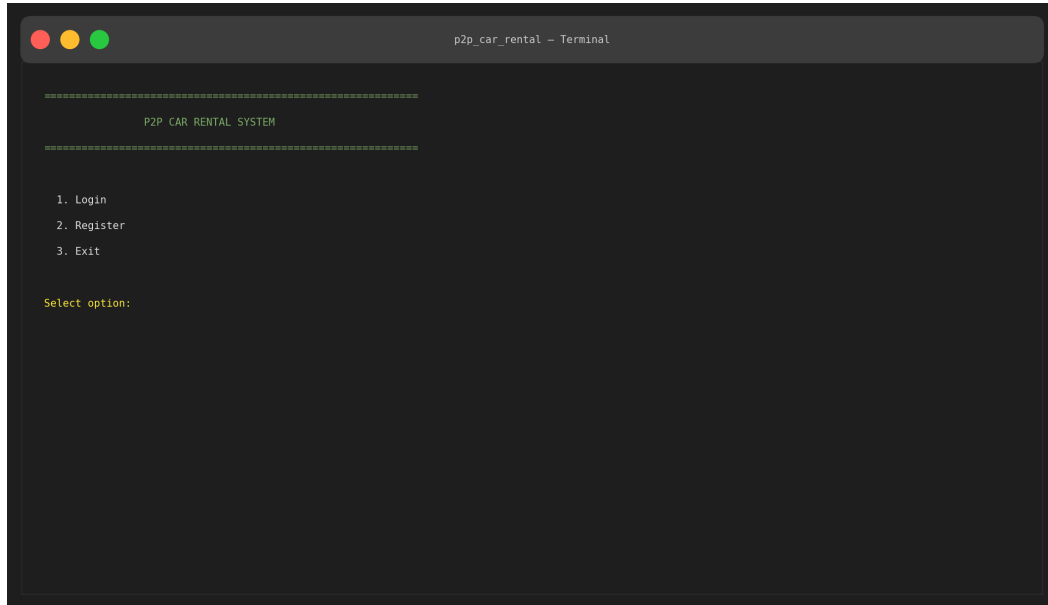


Figure 1: Main menu when the application starts.

8.2 Login

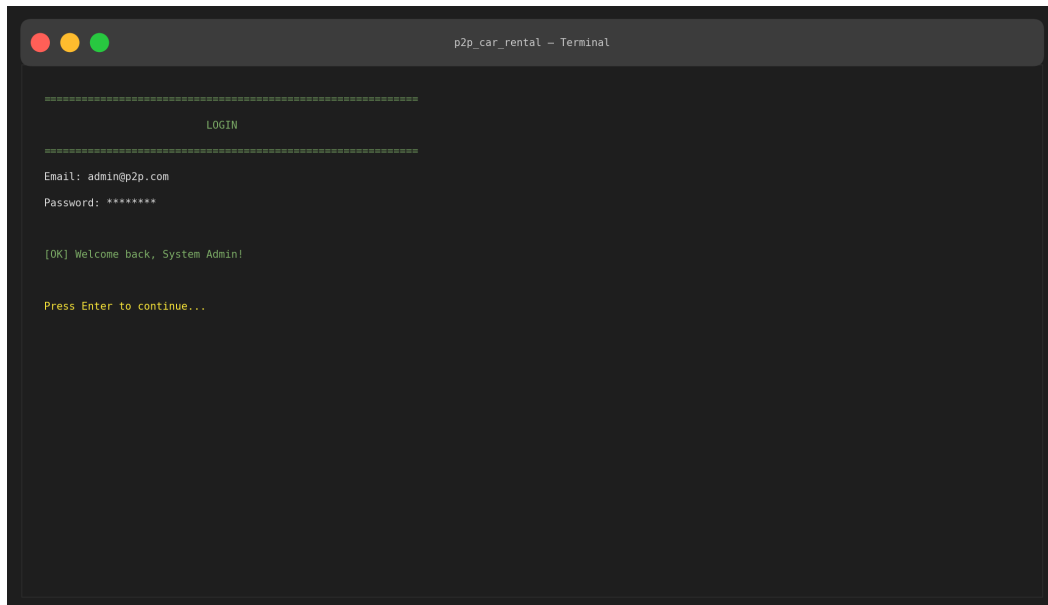


Figure 2: Login with a successful authentication.

8.3 Owner Dashboard

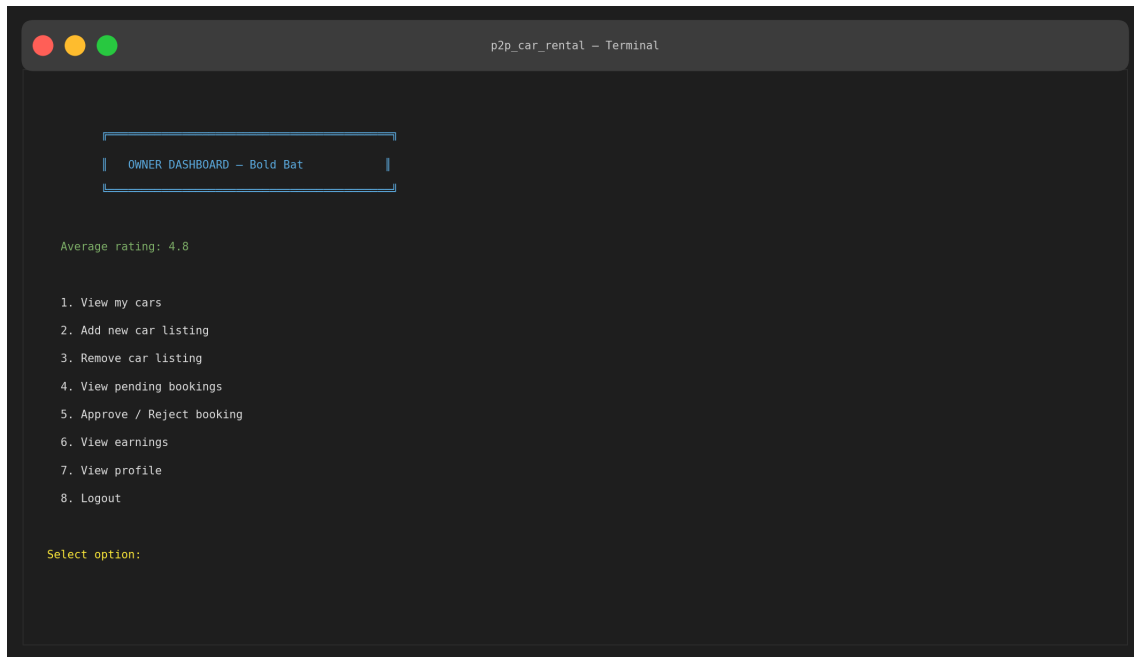


Figure 3: Owner dashboard with menu options.

8.4 Search Cars

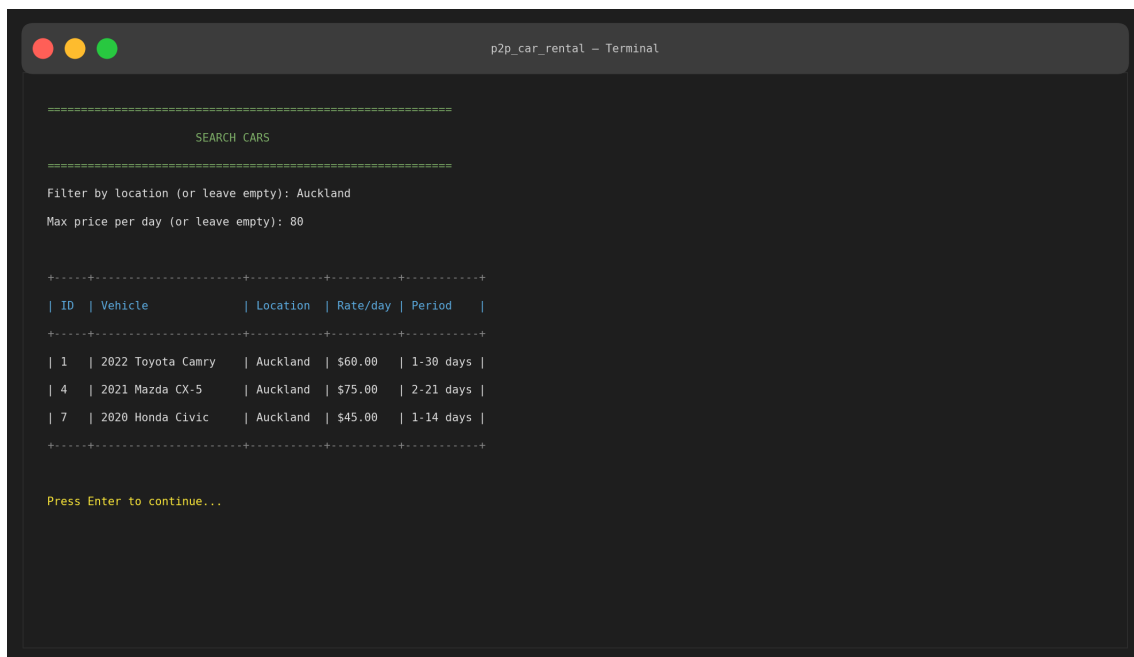
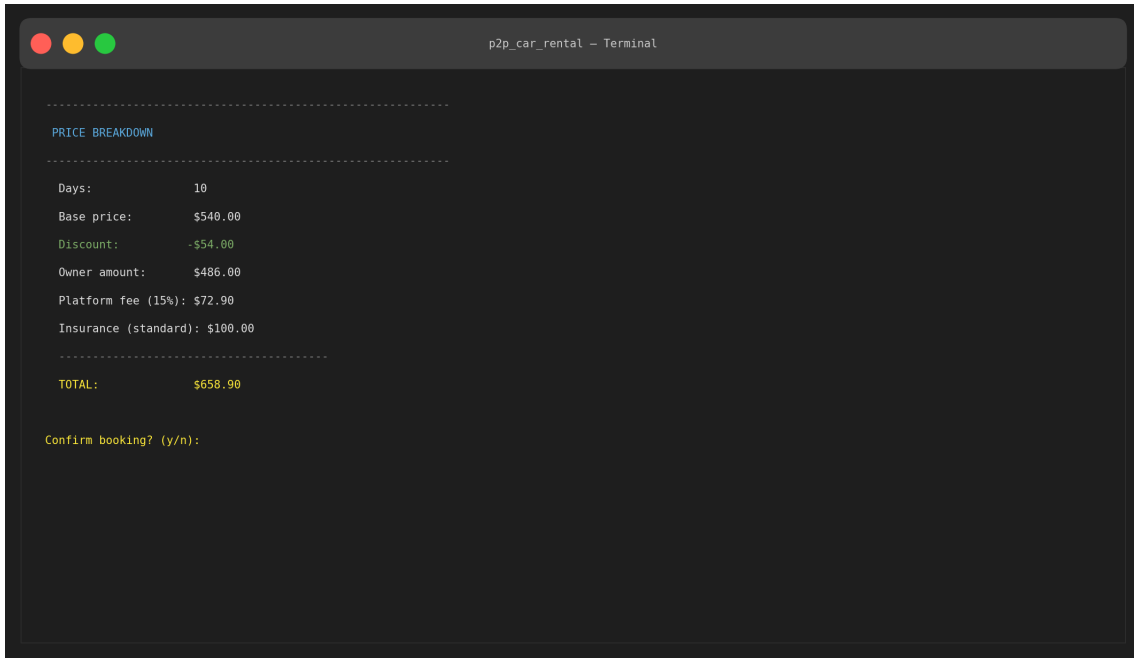


Figure 4: Renter searching for available cars.

8.5 Booking Price Breakdown



```
p2p_car_rental - Terminal

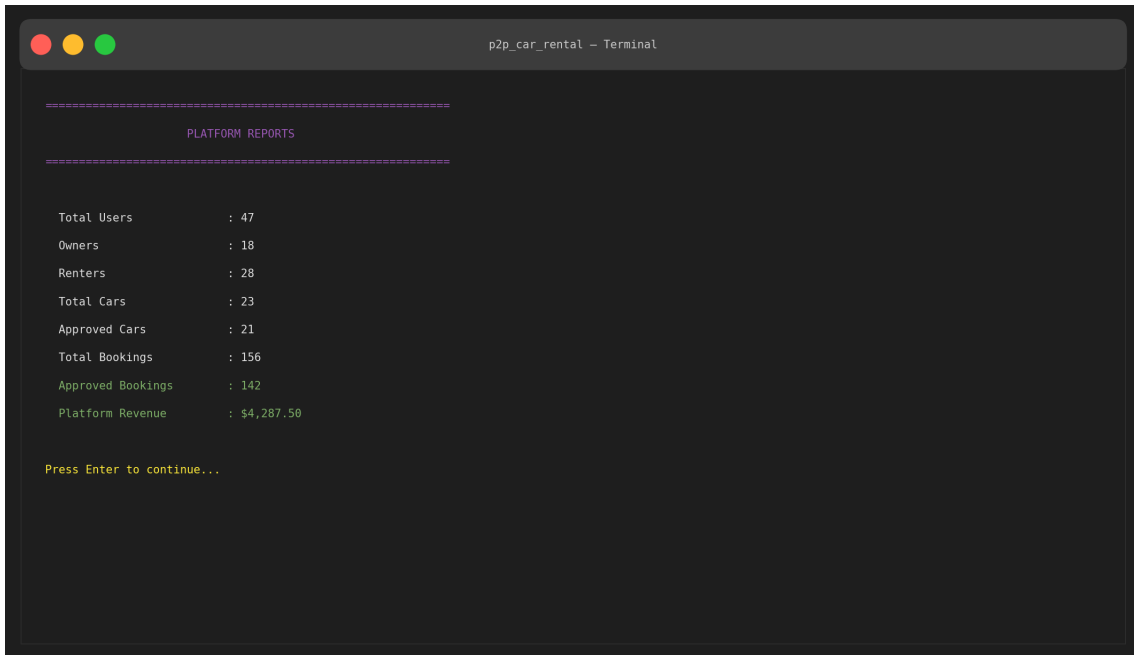
=====
PRICE BREAKDOWN
=====

Days:           10
Base price:     $540.00
Discount:       -$54.00
Owner amount:   $486.00
Platform fee (15%): $72.90
Insurance (standard): $100.00
=====
TOTAL:          $658.90

Confirm booking? (y/n):
```

Figure 5: Price breakdown shown before a booking is confirmed. The price includes a discount, insurance, and the platform fee.

8.6 Admin Platform Reports



```
p2p_car_rental - Terminal

=====
PLATFORM REPORTS
=====

Total Users      : 47
Owners           : 18
Renters           : 28
Total Cars       : 23
Approved Cars    : 21
Total Bookings   : 156
Approved Bookings : 142
Platform Revenue : $4,287.50

Press Enter to continue...
```

Figure 6: Admin reports showing platform statistics.

9. Design Patterns

The system uses three classical design patterns from the Gang of Four catalog (Gamma et al., 1994). Each pattern solves a specific problem in the code.

9.1 Singleton: DatabaseManager

Problem: If we open many database connections, we waste memory. We may also have problems when two parts of the code try to write at the same time.

Solution: Override `__new__` so that all calls to `DatabaseManager()` return the same object.

```
class DatabaseManager:
    __instance = None

    def __new__(cls):
        if cls.__instance is None:
            cls.__instance = super().__new__(cls)
            cls.__instance.__initialize()
        return cls.__instance
```

Benefit: Only one connection is used across the whole app. Tables are created only once.

9.2 Factory: UserFactory

Problem: When we load a user from the database, we need to create Owner, Renter, or Admin based on the role. If we use if/elif chains in many places, the code becomes hard to maintain.

Solution: Put all creation logic in `UserFactory.create(role, ...)`. If we need to add a new role, we only change one file.

```
class UserFactory:
    @staticmethod
    def create(role, user_id, name, email, ...):
        if role == 'owner':
            return Owner(user_id, name, email, ...)
        elif role == 'renter':
            return Renter(user_id, name, email, ...)
        elif role == 'admin':
            return Admin(user_id, name, email, ...)
```

9.3 Strategy: InsuranceStrategy

Problem: The system has three insurance plans (Basic, Standard, Premium). Each has a different daily fee and coverage. Using if/elif inside the booking service makes adding new plans difficult.

Solution: Each plan is a separate class that follows the InsuranceStrategy interface. The Booking has a reference and asks the plan to calculate the fee.

```
class InsuranceStrategy(ABC):
    @abstractmethod
    def calculate_fee(self, days): pass

class BasicInsurance(InsuranceStrategy):
    daily_fee = 5.0 # $5/day, $2000 coverage

class StandardInsurance(InsuranceStrategy):
    daily_fee = 10.0 # $10/day, $5000 coverage

class PremiumInsurance(InsuranceStrategy):
    daily_fee = 15.0 # $15/day, full coverage
```

Benefit: To add a new insurance plan, we only add a new class. The booking service does not change.

9.4 Pattern Summary

Pattern	Used for	File
Singleton	One database connection.	database/db_manager.py
Factory	Create User subclasses by role.	factories/user_factory.py
Strategy	Insurance pricing plans.	patterns/insurance_strategy.py

10. UML Diagrams

The system has 8 UML diagrams. They show the structure, behavior, and process flows. A short explanation is given after each diagram.

10.1 Entity Relationship Diagram (ERD)

The ERD shows the database schema. There are 5 tables with primary keys, foreign keys, and one unique key.

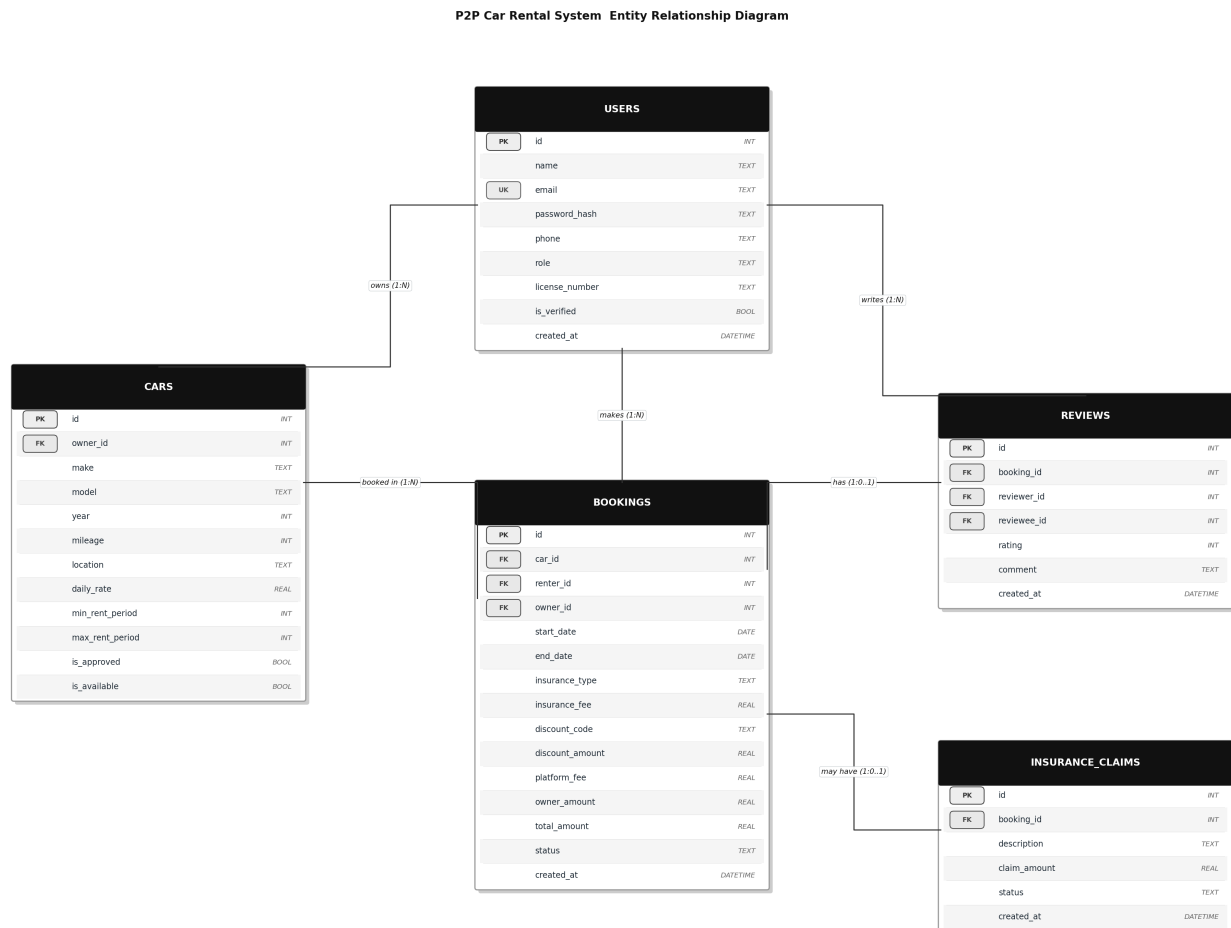


Figure 7: ERD with 5 tables, 8 foreign keys, and 1 unique key.

10.2 Use Case Diagram

This diagram shows what each user can do. It also shows "include" and "extend" relationships between use cases.

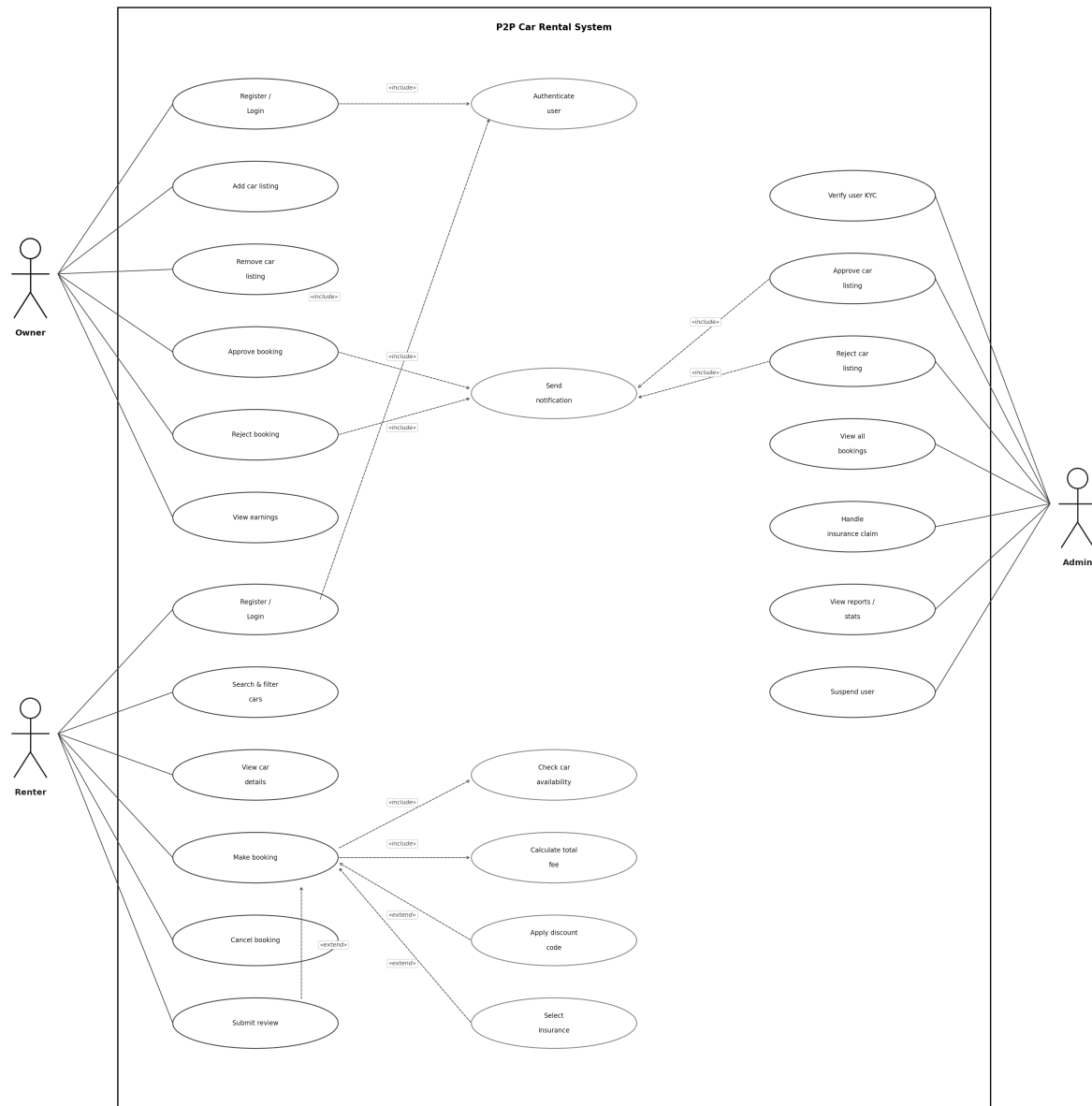


Figure 8: Use Case Diagram with 3 actors and 19 use cases.

10.3 Class Diagram

The class diagram shows the OOP design. It shows the abstract User base class, three subclasses, three pattern classes, and the entity classes.



Figure 9: Class Diagram showing the OOP hierarchy and patterns.

10.4 Sequence Diagram: Make Booking

This is the most complex use case in the system. The diagram shows the order of messages between objects.

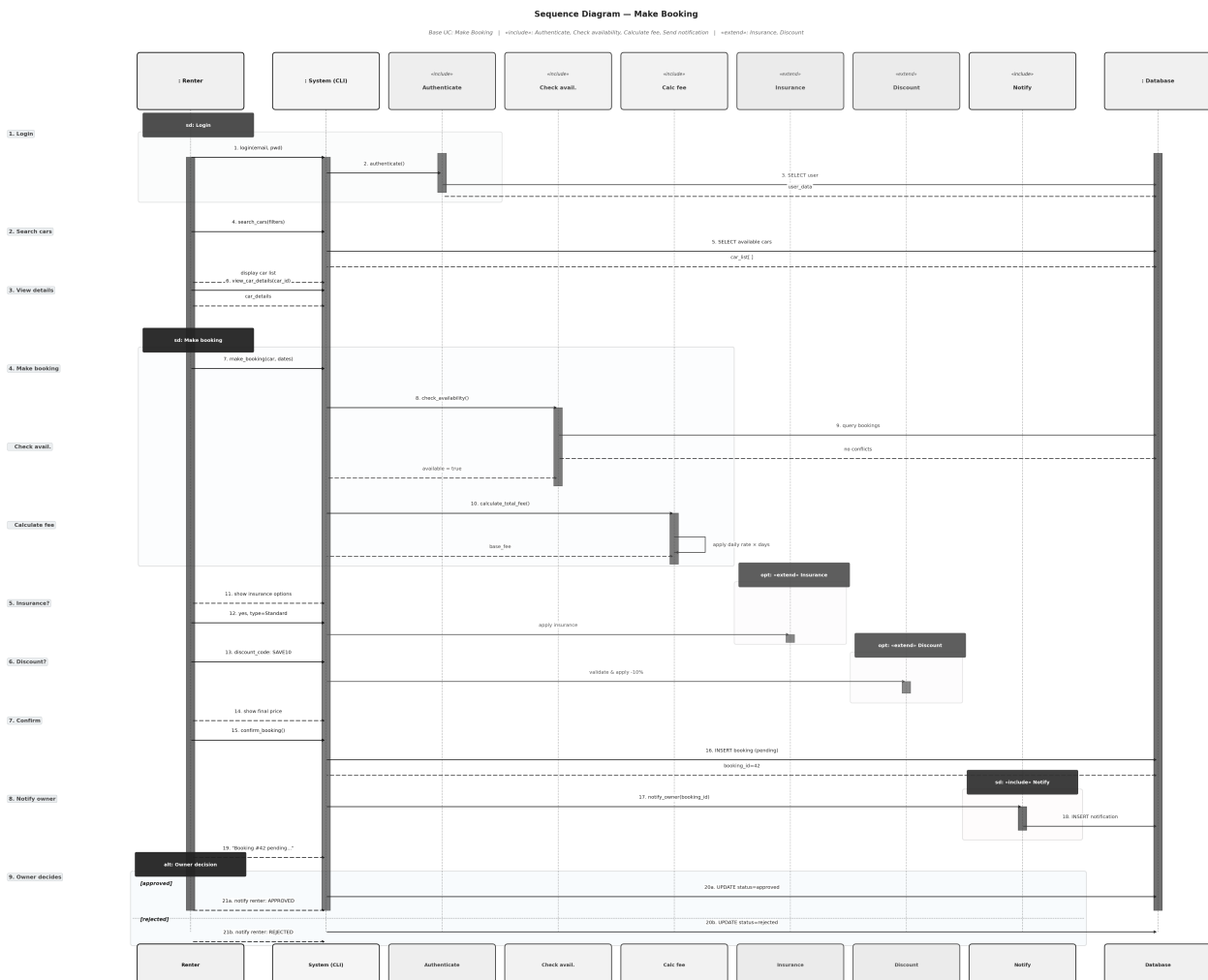


Figure 10: Sequence Diagram for the Make Booking use case.

10.5 Activity Diagram: Make Booking

This is the full process for a renter making a booking. It shows fork/join for parallel steps and optional extensions.

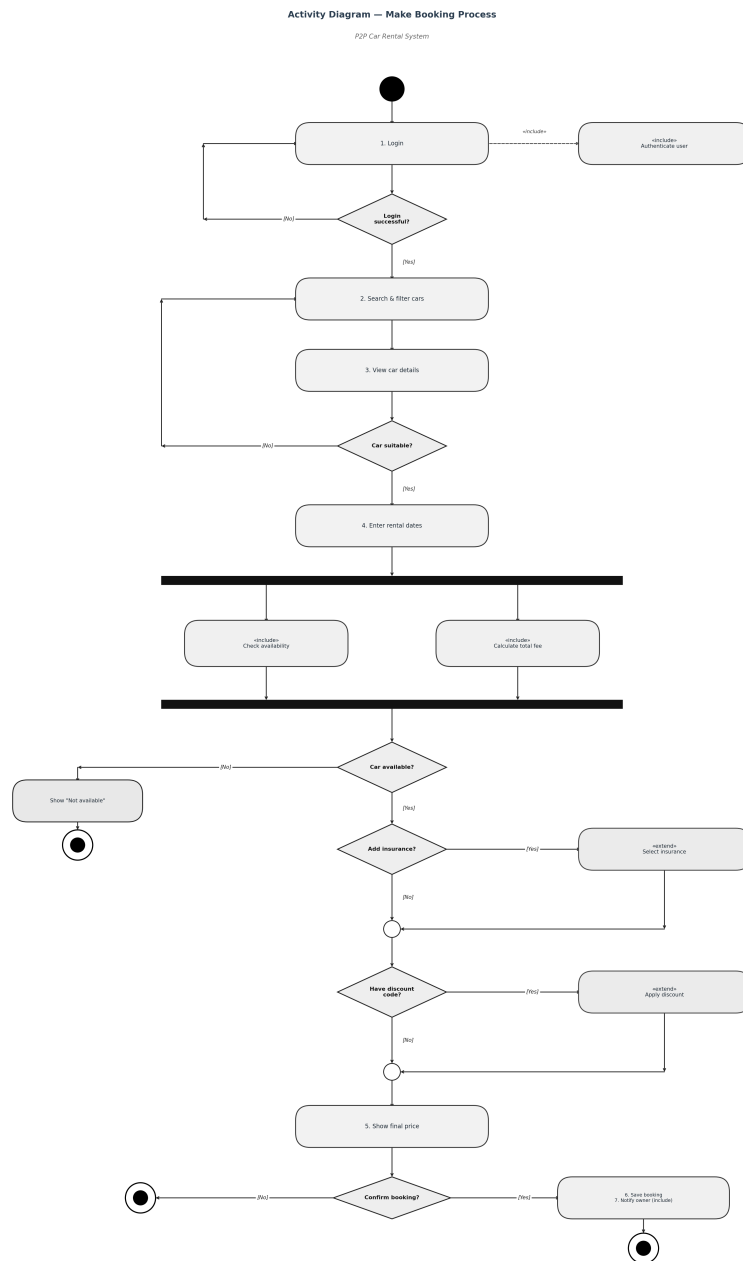


Figure 11: Activity Diagram for the Make Booking process.

10.6 Activity Diagram: Add Car Listing

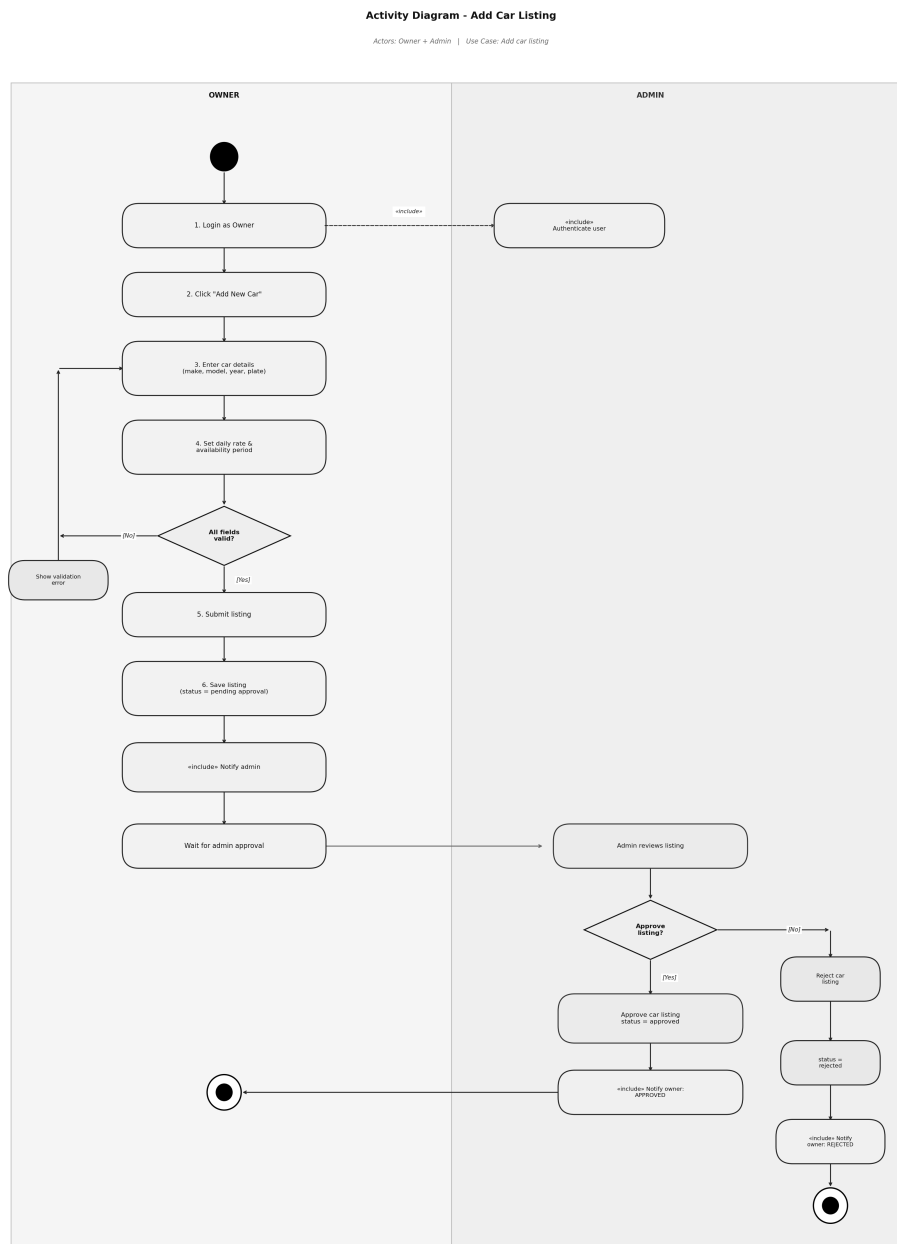


Figure 12: Activity Diagram for adding a car listing. The owner submits, and the admin approves.

10.7 Activity Diagram: Approve or Reject Booking

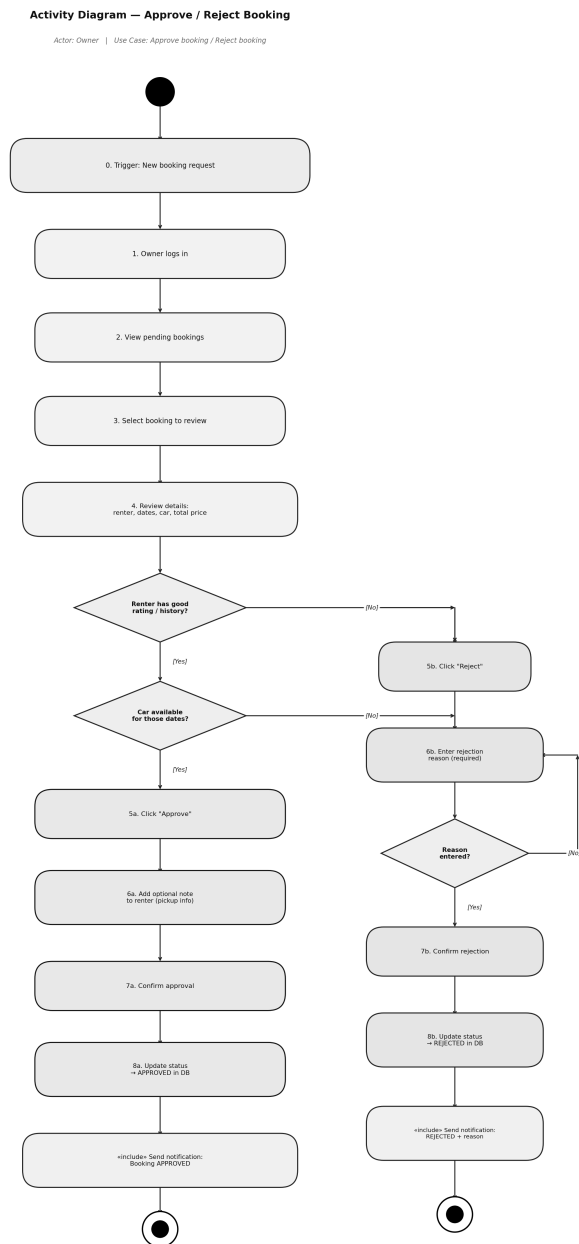


Figure 13: Activity Diagram for the owner approving or rejecting a booking.

10.8 Activity Diagram: Handle Insurance Claim

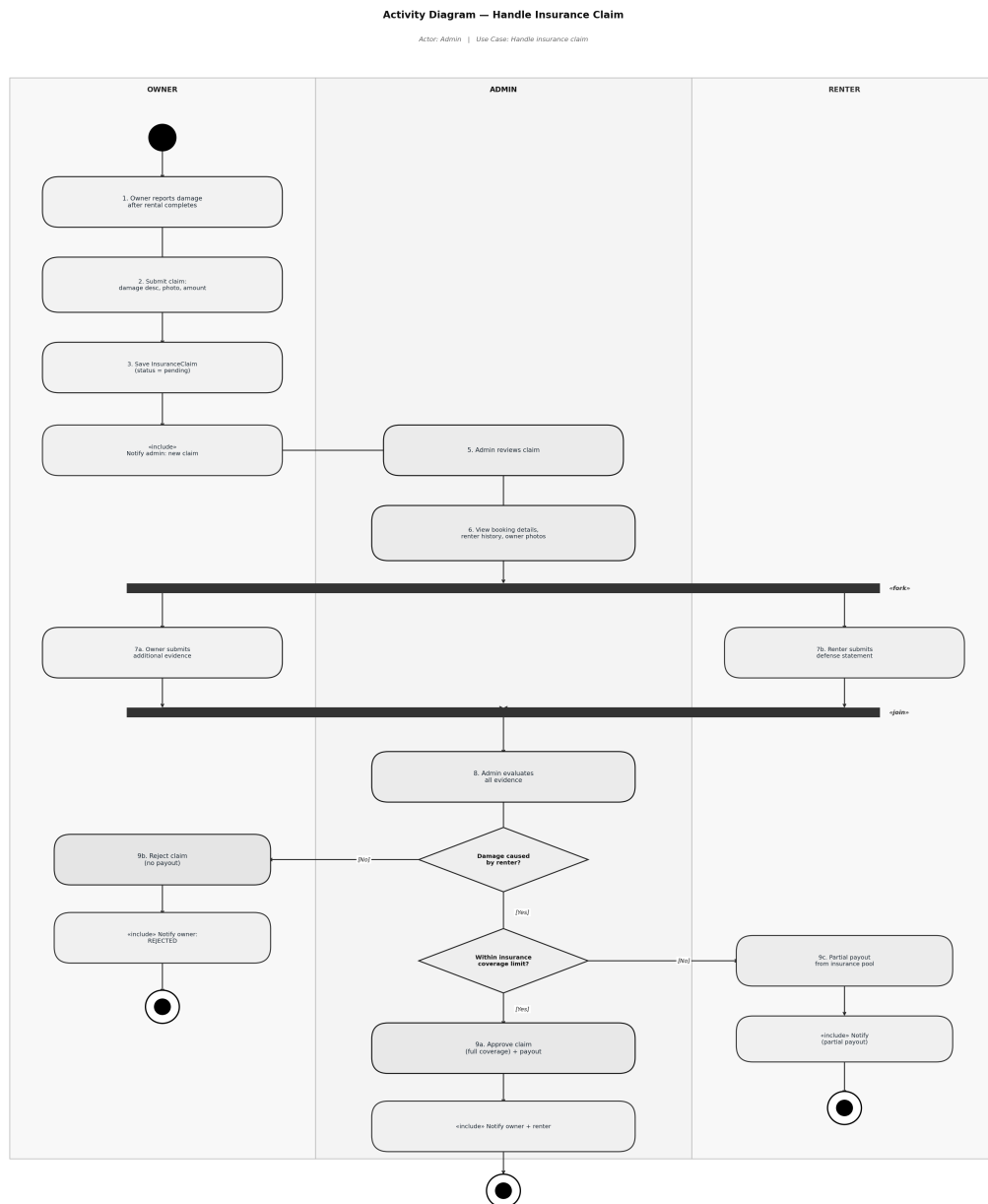


Figure 14: Activity Diagram for handling an insurance claim. It has 3 swimlanes and fork/join.

11. Testing

The system was tested by hand with integration tests. Each test checks a full feature from start to end.

11.1 Test Scenarios

#	Scenario	Result
1	Default admin is created on the first run.	PASS
2	Register an owner with a duplicate email.	PASS (rejected)
3	Admin verifies pending users.	PASS
4	Owner adds a car and admin approves it.	PASS
5	Renter searches by location and price.	PASS
6	Renter makes a 10-day booking with insurance and discount.	PASS
7	Pricing is calculated correctly.	PASS (see below)
8	Date overlap is detected (booking conflict).	PASS (rejected)
9	Owner approves a booking.	PASS
10	Renter submits a review (rating 1 to 5).	PASS
11	Renter submits an invalid rating (for example 6).	PASS (rejected)
12	Admin views platform statistics.	PASS
13	Login with a wrong password.	PASS (rejected)
14	Cancel a booking before the start date.	PASS
15	Cancel an already completed booking.	PASS (rejected)

11.2 Pricing Verification (Test #6)

A renter books a Toyota Camry at \$60 per day for 10 days. The renter selects Standard insurance and uses the SAVE10 discount code:

Calculation Step	Value
1. Base price (10 days x \$60)	\$600.00
2. Long-term discount (7+ days = 10% off)	-\$60.00
3. Subtotal	\$540.00

Calculation Step	Value
4. SAVE10 discount code (-10%)	-\$54.00
5. Owner amount	\$486.00
6. Platform fee (15% of owner amount)	+\$72.90
7. Standard insurance (\$10/day x 10 days)	+\$100.00
8. TOTAL paid by renter	\$658.90

The output of the system matches this manual calculation. The pricing engine works correctly.

12. Maintenance Summary

A full Maintenance and Support Plan is in a separate document. This section is a short summary of the three main strategies.

12.1 Software Maintenance

The plan covers four types of maintenance from ISO/IEC 14764 (ISO/IEC, 2006): corrective (bug fixes), adaptive (environment changes), perfective (improvements), and preventive (avoid future problems). The plan has a 9-step process to handle issues and a 4-level priority system.

12.2 Versioning

We use Semantic Versioning 2.0.0 (Preston-Werner, 2013). The format is MAJOR.MINOR.PATCH. Each release is tagged in Git and recorded in CHANGELOG.md (Oliver, 2017).

12.3 Backward Compatibility

Three pillars: data compatibility (migration scripts), API compatibility (stable function names), and user experience (similar workflows). Old features are removed in three stages (announce, coexist, remove) over at least 6 months.

For full details, see the separate document: *Maintenance_and_Support_Plan.pdf*

13. Conclusion

This project shows how OOP principles, design patterns, and a layered architecture can work together in a small Python project.

13.1 What Was Done

- All four OOP principles are used in working code, not just in theory: encapsulation, inheritance, abstraction, and polymorphism.
- Three design patterns (Singleton, Factory, Strategy) are used where they solve real problems.
- A 4-layer architecture separates presentation, business logic, domain entities, and data access.
- Configuration supports dev, test, and production environments.
- 8 UML diagrams document the system structure and behavior.
- A full Maintenance and Support Plan is provided.

13.2 Lessons Learned

During this project, I learned a few important things:

- Choosing a layered architecture early made adding new features (insurance, discount codes, reviews) much easier.
- Design patterns are tools, not goals. Each pattern was used because it solved a real problem.
- Documentation is important. UML diagrams and good comments help future maintainers understand the code.
- Externalizing configuration is very useful. The APP_ENV variable made testing much easier.

13.3 Future Work

The Maintenance Plan has a 24-month roadmap. Some highlights:

- Email notifications by SMTP (v1.1.0, Q3 2026).
- Loyalty points and tier discounts (v1.2.0, Q4 2026).
- Multi-language support (v1.3.0, Q1 2027).
- PostgreSQL backend and REST API (v2.0.0, Q2 2027).
- Mobile app with React Native (v2.1.0, Q3 2027).
- Stripe payment integration (v2.2.0, Q4 2027).
- AI-based smart matching (v3.0.0, Q2 2028).

References

- Gamma, E., Helm, R., Johnson, R., and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley.
- ISO/IEC (2006). *ISO/IEC 14764:2006 Software Engineering - Software Life Cycle Processes - Maintenance*. Geneva: International Organization for Standardization.
- Oliver, O. (2017). *Keep a Changelog 1.0.0*. Retrieved from <https://keepachangelog.com/>
- Preston-Werner, T. (2013). *Semantic Versioning 2.0.0*. Retrieved from <https://semver.org/>

Appendix: Default Credentials

On the first run, the system creates a default admin account. You can use this account to verify the first new users.

Field	Value
Email	admin@p2p.com
Password	admin123
Role	Admin (pre-verified)

Security note: Change the default admin password before using the system in production. The credentials above are for development and demonstration only.

Document Reference

Document	Purpose
Final_Report.pdf (this document)	Full overview of the project.
Design_and_Architecture.pdf	UML diagrams in a separate PDF.
Maintenance_and_Support_Plan.pdf	Detailed maintenance strategy.
README.md	Quick-start instructions.
p2p_car_rental.zip	Full project source code.